SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1709**

Register Number	**192311366**
Name	**Gondel Prashanth**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.*(BL3)*

Assessment Tool Weightage: 10%

QUESTIONS:2 **Total Marks:20**

Descriptive Test

1. A smart home automation system is being designed to manage lighting, temperature, and security based on user behaviour and environmental conditions. The system must respond to real-time inputs such as motion detection, temperature changes, and time of day while also learning user preferences over time. In this context, explain the different types of intelligent agents (simple reflex, model-based, goal-based, and utility-based agents) and analyze how each type would function within this system. Justify which type of agent or hybrid approach would be most effective for ensuring comfort, energy efficiency, and security.

2. A robot is placed in an unknown maze and must find a path to the exit without prior knowledge of the layout. Explain how uninformed search strategies like Uniform Cost Search (UCS) and Iterative Deepening Search (IDS) can help solve this problem.

   Discuss the advantages and disadvantages of these algorithms in terms of time and space complexity. Relate the problem to different types of intelligent agents and explain how agent design affects decision-making. Suggest which combination of agent type and search strategy would be most effective and justify your choice.

Code of Conduct:

I Gondel Prashanth / 192311366 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criterion	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Max Marks
Criteria	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**	**Max Marks**
Understanding of Concepts	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions	5
Explanation	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation	5
Application	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis	4
Organization	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers	3
Accuracy	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps	2
Clarity	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand	1

Anwers for above Questions:

1)

i. Simple Reflex Agents:

These agents make decisions based exclusively on the current real-time input ignoring any past history. They operate on basic "if-then" condition-action rules.

- How it functions:
  - Lighting: if motion is detected, then turn on the lights. IF no motion is detected, then turn off the lights.
  - Temperature: if temperature is greater than 25°C, then turn on the AC.
- Strengths: Incredibly fast and requires minimal computing power. Great for immediate hazard responses (e.g., if smoke detected, then sound alarm).
- Weaknesses: It has no memory. If you are sitting perfectly still reading a book, a simple reflex agent will assume the room is empty and turn the lights off, causing frustration.

ii. Model-Based Reflex Agents:

These agents maintain an internal "model" or memory of the world. They combine current sensor data with past history to understand the unobserved parts of their environment.

- How it functions:
  - Lighting: The system tracks that motion was detected in the living room, but the living room door hasn't opened since. Therefore, its internal model states the user is still in the room. It keeps the lights on even if the user is sitting still.
  - Security: It knows the time of day and that the front door was unlocked with an authorized PIN. It updates its state to "Owner Home" and disables the interior motion alarms, rather than blindly triggering an alarm upon seeing motion.
- Strengths: Handles partial observability and context beautifully.
- Weaknesses: It doesn't plan for the future or optimize for preferences; it merely reacts accurately to the current state of the house.

iii. Goal-Based Agents:

A goal-based agent plans its actions by considering future consequences to achieve a specific target state. It asks, "What will happen if I do this, and does it get me closer to my goal?"

- How it functions:
  - Temperature: Goal = "Ensure the house is 22°C by the time the user arrives from work at 6:00 PM." The agent calculates how long it takes to cool the house and might turn on the AC at 5:15 PM to reach that specific goal.
  - Security: Goal = "Secure the perimeter for the night." The agent sequences multiple actions: close the smart blinds, lock the deadbolts, arm the external cameras, and turn on the porch light.
- Strengths: Proactive rather than purely reactive. Excellent at executing complex, multi-step routines.
- Weaknesses: It is binary. A goal is either achieved or it isn't. It doesn't know how to handle conflicting goals (e.g., cooling the house to exactly 22°C might conflict with an energy-saving goal).

iv. Utility-Based Agents:

When there are multiple goals or ways to achieve a goal, a utility-based agent measures how "happy" or "efficient" a state is. It evaluates trade-offs to maximize a numerical utility score .

- How it functions:
    - Comfort vs. Energy: It is a warm afternoon, and electricity prices are currently at peak rates. The agent calculates that running the ceiling fans and closing the blinds (low energy cost, moderate comfort) yields a higher overall "utility score" than blasting the AC (high energy cost, high comfort).
    - Learning Preferences: If the user manually overrides the system to turn on the AC anyway, the agent updates its utility function, learning that this specific user heavily prioritizes comfort over cost during the afternoon.
- Strengths: Highly adaptable. It makes nuanced compromises and optimizes for long-term satisfaction and efficiency.

2)

i). How UCS and IDS Solve the Maze:

Uninformed search algorithms also known as blind searches do not have a heuristic—meaning they have no sense of which direction is "closer" to the exit. They only know what they have explored so far.

- Uniform Cost Search : UCS expands paths based on the lowest cumulative cost. If every step in the maze costs the same, UCS behaves exactly like Breadth-First Search, expanding outward in a uniform radius. If the maze has varied terrain , UCS prioritizes the cheapest overall path.
- Iterative Deepening Search : IDS combines the memory efficiency of Depth-First Search with the completeness of BFS. It explores the maze by setting a depth limit (e.g., "explore everything 1 step away"). If the exit isn't found, it increases the limit to 2 steps, starts over, and goes deeper. While starting over sounds wasteful, the tree grows exponentially, meaning the vast majority of the work is done in the final layer anyway.

ii)Complexity Analysis: UCS vs. IDS:

When evaluating these algorithms, we look at the branching factor (b), depth of the shallowest goal (d), maximum depth (m), the cost of the optimal solution (C*), and the minimum step cost (ϵ).

Uniform Cost Search (UCS)

- Time Complexity: O(b^(1+[c*/E]))
- Space (Memory) Complexity: O(b^(1+[c*/E]))
- Advantages: It is optimal and complete. It is guaranteed to find the absolute cheapest path to the exit if a path exists.
- Disadvantages: It has a massive memory cost. Because UCS must keep all explored nodes and the entire search frontier in memory, it will quickly run out of RAM in a large maze.

Iterative Deepening Search (IDS)

- Time Complexity: O(b^d)
- Space (Memory) Complexity: O(bd)
- Advantages: It is highly memory efficient. By only storing the current path being explored, the memory requirement grows linearly rather than exponentially. It is also complete and optimal, provided all steps in the maze cost the same.

- Disadvantages: It performs redundant work by revisiting the upper nodes of the maze multiple times. Furthermore, it cannot guarantee an optimal path if the steps have varying costs.

iii. Intelligent Agents and Decision-Making:

An algorithm is just the math; the agent design determines how the robot actually interacts with the maze.

- Simple Reflex Agent: Makes decisions based only on the current sensor reading (e.g., "If wall ahead, turn left"). Effect on decision-making: It will easily get trapped in infinite loops or dead ends because it has no memory of where it has been.
- Model-Based Reflex Agent: Keeps an internal state (a mental map) of what it has seen so far. Effect on decision-making: It won't get stuck in loops because it remembers dead ends, but it only reacts to the current situation rather than planning a route to the exit.
- Goal-Based Agent: Uses its internal model to plan sequences of actions that lead to a specific target. Effect on decision-making: When it discovers a new path, it can pause, search its mental map, and plan a sequence of steps toward a known frontier or the exit.
- Utility-Based Agent: Similar to a goal-based agent, but it measures how "good" a path is. Effect on decision-making: If there are two ways out, it will evaluate the trade-offs (e.g., shortest distance vs. battery consumption) to pick the optimal route.

iv. The Most Effective Combination:

The best approach for this scenario is a Model-Based Goal Agent using Iterative Deepening Search .

Justification:

- In an unknown environment, the robot must have an internal model to map the maze as it explores. Without memory, it is impossible to navigate a complex maze without looping. The goal-based design allows it to dynamically plan paths to unvisited areas or the exit as its map grows.
- IDS In physical robotics, memory (RAM) is often tightly constrained. Because the maze is unknown, an algorithm like UCS requires storing the entire frontier of the search, leading to an exponential space footprint. IDS provides a linear memory footprint while still guaranteeing that the robot will find the shortest path (assuming standard 1-unit step costs).